

Reciprocal Rank Fusion (RRF), Worked Out by Hand

The production default for combining retrievers — it throws away the incomparable scores and fuses on *rank position* alone, $\sum 1/(k+\text{rank})$. Derived from scratch and computed on two ranked lists.

What this document does. It motivates why fusing ranks beats fusing scores, defines the RRF formula, computes the fused ranking of five documents from a BM25 ordering and a dense ordering by hand, and explains the role of the constant k (usually 60). It contrasts RRF with the score-based convex combination of #20. Numbers from Appendix A.

Context: the alternative path was min-max (#19) + convex combination (#20). RRF is the robust shortcut that skips both.

1 The idea: fuse ranks, not scores

Score-based fusion (#19–#20) is fragile: BM25 and cosine scores live on different, outlier-prone, per-query scales, so normalising them is delicate. RRF sidesteps the whole problem by discarding magnitudes and keeping only what is robust — *the position* each retriever assigned. A document ranked #1 by BM25 and #3 by dense is described purely by “1” and “3,” never by 8.2 and 0.55.

Intuition

RRF runs an election. Each retriever submits a ranked ballot; a document earns points for *where* it lands on each ballot — a lot for the top spot, sharply less further down. Sum each document’s points across ballots and re-sort. Because points depend only on rank, no retriever’s idiosyncratic score scale can stuff the ballot box. Consensus near the top of *either* list wins; this is why RRF is the boring, reliable default.

2 The formula

For a document d ranked at position $r_j(d)$ by retriever j (rank 1 = best), with a constant k :

$$\text{RRF}(d) = \sum_j \frac{1}{k + r_j(d)}$$

The contribution $1/(k + r)$ is largest at rank 1 and decays with rank — but gently, because k (typically 60) dominates the denominator. Missing from a list simply contributes 0 (or a small floor). Higher total = better.

Why the $+k$? Without it, rank 1 would score $1/1 = 1.0$ and rank 2 only $1/2 = 0.5$ — the top spot would crush everything, so a single retriever’s #1 could never be overruled. With $k = 60$, rank 1 scores $1/61 = 0.0164$ and rank 2 scores $1/62 = 0.0161$: top positions are close, so *agreement across retrievers* outweighs a lone strong opinion. k flattens the curve and controls how much being #1 on one list beats being #2 on two.

3 Worked by hand

Two retrievers return these orderings of documents A – E :

BM25: $[A, C, B, E, D]$, dense: $[B, D, A, C, E]$.

So the ranks are:

doc	r_{BM25}	r_{dense}	$\text{RRF} = \frac{1}{60+r_{\text{BM25}}} + \frac{1}{60+r_{\text{dense}}}$
A	1	3	$\frac{1}{61} + \frac{1}{63} = 0.016393 + 0.015873 = 0.032266$
B	3	1	$\frac{1}{63} + \frac{1}{61} = 0.015873 + 0.016393 = 0.032266$
C	2	4	$\frac{1}{62} + \frac{1}{64} = 0.016129 + 0.015625 = 0.031754$
D	5	2	$\frac{1}{65} + \frac{1}{62} = 0.015385 + 0.016129 = 0.031514$
E	4	5	$\frac{1}{64} + \frac{1}{65} = 0.015625 + 0.015385 = 0.031010$

Fused ranking: $A = B$ (0.03227) > C (0.03175) > D (0.03151) > E (0.03101).

Mini-example • consensus beats a lone #1

A is BM25's #1 (and dense's #3); B is dense's #1 (and BM25's #3). They tie at the top — symmetric evidence. Now look at C : never #1, but ranked #2 and #4 — *consistently near the top of both*. It lands third overall, ahead of D , which scored a #2 but also a lowly #5. RRF rewards C 's steady cross-retriever agreement over D 's one strong-one-weak split. That is the election working as intended.

4 RRF vs. convex combination

	Convex combination (#20)	RRF (#21)
input	normalised scores	ranks only
needs normalisation?	yes (min-max, fragile)	no
outlier-sensitive?	yes	no
tunable?	α (trust weight)	k (rank flattening)
keeps score margins?	yes	no (discarded)

RRF wins by default because it is parameter-light and robust: no per-query rescaling, no outlier blow-ups, and it fuses any number of retrievers (even non-comparable ones like BM25 + dense + a graph search) uniformly. Convex combination wins only when the score *margins* carry real information you do not want to throw away — rare in practice, which is why RRF is the production standard.

Equation cheat-sheet

$$\text{RRF}(d) = \sum_j \frac{1}{k + r_j(d)}, \quad k \approx 60, \quad \text{rank } 1 \rightarrow \frac{1}{k+1}$$

no score normalisation; robust to outliers; larger $k \Rightarrow$ flatter (consensus matters more)

Appendix A • Reference implementation

```
def rrf(rankings, k=60):
    # rankings: list of ordered doc lists
    scores = {}
    for ranked in rankings:
        for rank, doc in enumerate(ranked, start=1):
            scores[doc] = scores.get(doc, 0) + 1 / (k + rank)
    return sorted(scores.items(), key=lambda kv: -kv[1])
```

```
rrf(["A","C","B","E","D"], ["B","D","A","C","E"])  
# A 0.032266, B 0.032266, C 0.031754, D 0.031514, E 0.031010  ->  A=B>C>D>E
```

Internal learning material. Numbers reproducible from the appendix code. v1.0